

Predicting Customer Churn with Machine Learning

An end-to-end project with XGBoost, SVM, Logistic Regression & Business Insights

The Business Problem

Imagine you run a telecom company and roughly **1 out of every 4 customers leaves you every year**. Each one of those lost customers costs you between \$200–\$400 just to replace them — and that's before you account for the revenue you lost while they were gone.

The question is: **can we predict who is about to leave before they actually do?**

If we can identify those customers in advance, the company can intervene — offer a discount, a better plan, a call from customer service — and potentially save that relationship before it's too late. That's the exact problem machine learning is built to solve.

What You Will Learn

By working through this notebook, you will understand:

- How to **explore and understand a real business dataset** before touching any model
 - How to **clean and prepare data** the right way, avoiding common mistakes like data leakage
 - How to **build and evaluate three different ML models**: XGBoost, SVM, and Logistic Regression
 - How to **explain model predictions** using SHAP values — not just what the model says, but *why*
 - How to **compare models** and decide which one to use in a real-world context
-

Why Machine Learning for This?

A human analyst could look at a few hundred customers and make educated guesses about who might leave. But with thousands of customers and dozens of variables per person, the patterns are too complex to spot manually. Machine learning algorithms can process all of that data simultaneously and find combinations of signals that no human would catch — like the fact that a customer on a month-to-month contract, with high monthly charges, and less than 12 months of tenure is far more likely to churn than anyone else.

There are many classification models available (Random Forest, Neural Networks, Logistic Regression, SVM, XGBoost...). In this notebook we explore three of them so you can see how they compare against each other.

Project Methodology

Every serious ML project follows a structured workflow. Here's ours:

Step	What happens	Why it matters
1. EDA	Explore and understand the data	You can't model what you don't understand
2. Basic Cleaning	Fix types, drop irrelevant columns	Garbage in = garbage out
3. Train/Test Split	Divide data 70/30 <i>before</i> preprocessing	Prevents data leakage
4. Full Preprocessing	Encode categories, handle outliers	Fit only on train, apply to test
5. Modeling	Train the algorithms	The actual machine learning
6. Evaluation	Measure performance with the right metrics	Accuracy alone is misleading
7. Model Comparison	Compare all three models	Choose the right tool for the job

Key principle — Data Leakage: We always split the data *before* doing any statistics-based preprocessing (like encoding or scaling). If we did it the other way around, the model would have "seen" information from the test set during training, making our results unrealistically optimistic. Think of it like a student memorizing the exam answers — the grade looks great, but the learning didn't actually happen.

Section 1 — Setup & First Look at the Data

Before we can build anything, we need to load our tools and take a first look at what we're working with. This is not optional — skipping this step is one of the most common beginner mistakes in ML. You need to understand your data's structure, size, and quality before you can make any meaningful decisions about how to process it.

```
# --- Library imports ---
# pandas and numpy handle all our data manipulation
# matplotlib and seaborn are our visualization tools
# shap lets us explain why the model made each prediction
# sklearn provides the split, encoding, metrics, SVM, and Logistic Regression tools
# xgboost is our primary model
```

```

from google.colab import drive
drive.mount('/content/drive')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import shap
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import classification_report, roc_auc_score,
ConfusionMatrixDisplay, RocCurveDisplay
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.decomposition import PCA
from xgboost import XGBClassifier

# Load the dataset
# This is the Telco Customer Churn dataset – publicly available on
Kaggle
# It contains 7,043 customers and 21 variables about their service and
behavior
df = pd.read_csv('/content/drive/MyDrive/Eproducts/WA_Fn-UseC_-Telco-
Customer-Churn.csv')

# .info() shows column names, data types, and how many non-null values
exist per column
# This is our first health check – we want to spot any missing data or
wrong types immediately
df.info()
df.head(5)

Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID             7043 non-null   object
1   gender                 7043 non-null   object
2   SeniorCitizen          7043 non-null   int64
3   Partner                7043 non-null   object
4   Dependents             7043 non-null   object
5   tenure                 7043 non-null   int64
6   PhoneService           7043 non-null   object
7   MultipleLines           7043 non-null   object
8   InternetService        7043 non-null   object
9   OnlineSecurity         7043 non-null   object
10  OnlineBackup            7043 non-null   object

```

```
11 DeviceProtection 7043 non-null object
12 TechSupport      7043 non-null object
13 StreamingTV      7043 non-null object
14 StreamingMovies  7043 non-null object
15 Contract         7043 non-null object
16 PaperlessBilling 7043 non-null object
17 PaymentMethod    7043 non-null object
18 MonthlyCharges   7043 non-null float64
19 TotalCharges     7043 non-null object
20 Churn            7043 non-null object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

```
{"type": "dataframe", "variable_name": "df"}
```

We have **7,043 rows and 21 columns**. Notice that `TotalCharges` is listed as `object` (text) instead of a number — that's a red flag we'll fix later. Everything else looks as expected for now.

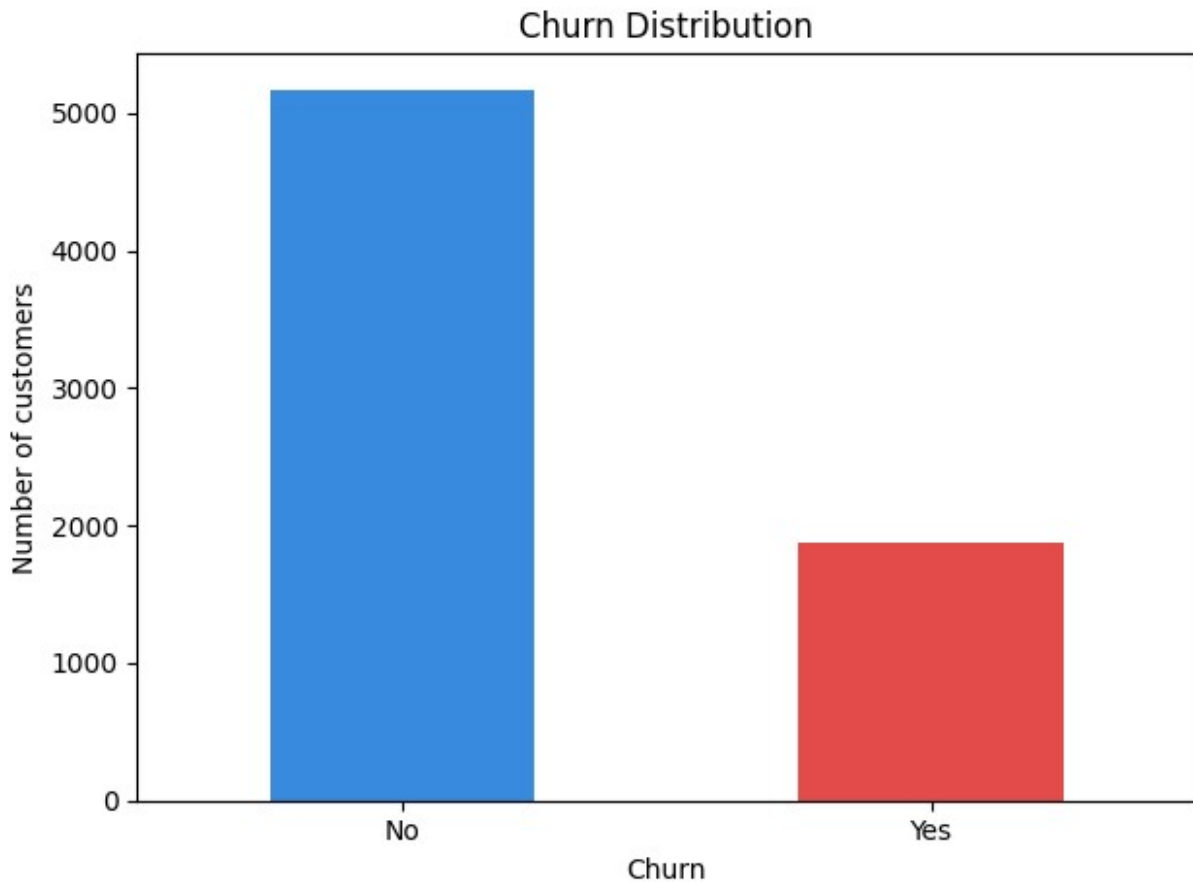
Section 2 — Exploratory Data Analysis (EDA)

EDA is the process of visually and statistically exploring the data to find patterns, relationships, and anomalies. Think of it as reading the story the data is trying to tell you before you ask the model to learn it.

2.1 — How many customers actually churn?

```
# A bar chart of churners vs. non-churners
# This immediately tells us how balanced (or imbalanced) our target
variable is
# An imbalanced dataset – say 90% stays vs 10% leaves – requires
special handling
# because a model that always predicts 'stays' would look 90% accurate
but be useless
```

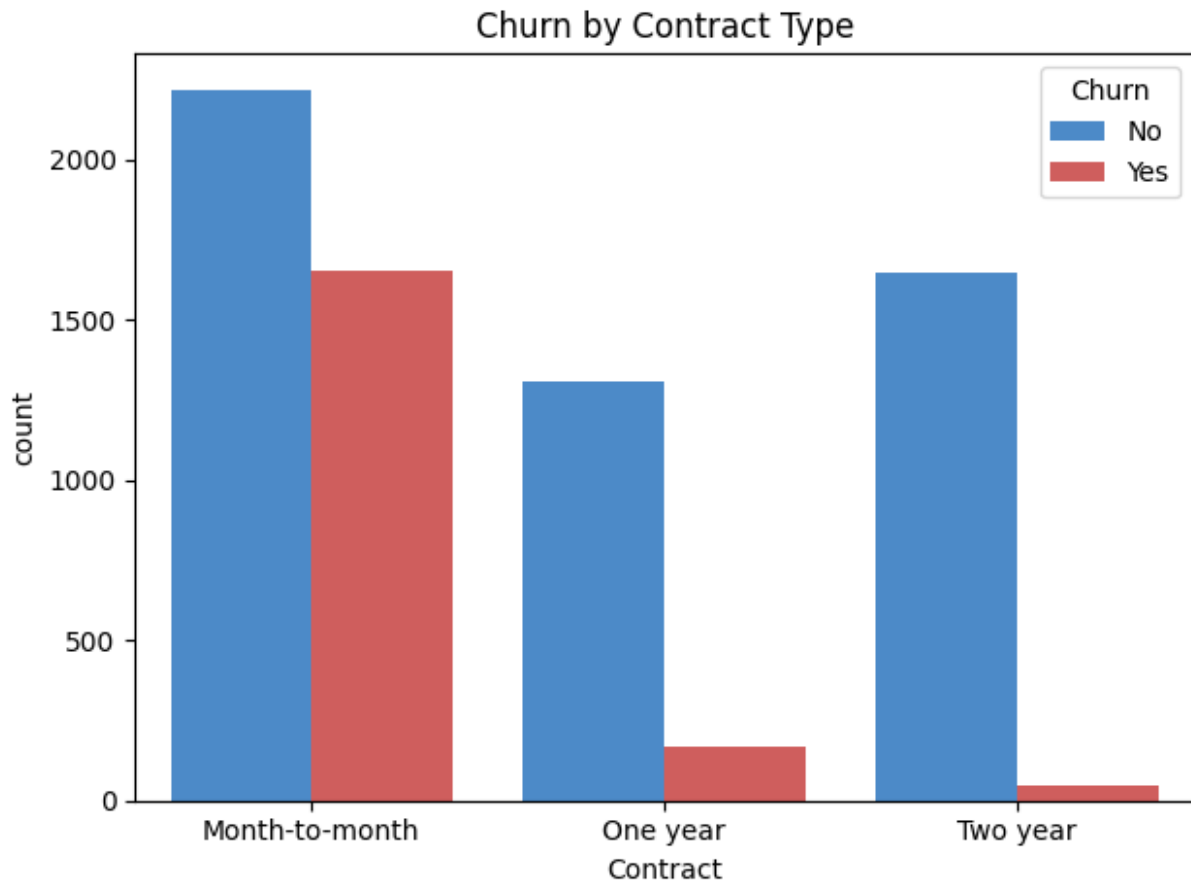
```
ax = df['Churn'].value_counts().plot(kind='bar',
color=['#378ADD', '#E24B4A'])
ax.set_title('Churn Distribution')
ax.set_ylabel('Number of customers')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



About **73% of customers stay** and **27% churn**. This is a moderately imbalanced dataset — not extreme, but significant enough that we need to account for it in our models (which we'll do using `scale_pos_weight` for XGBoost and `class_weight='balanced'` for SVM and Logistic Regression).

2.2 — Does contract type predict churn?

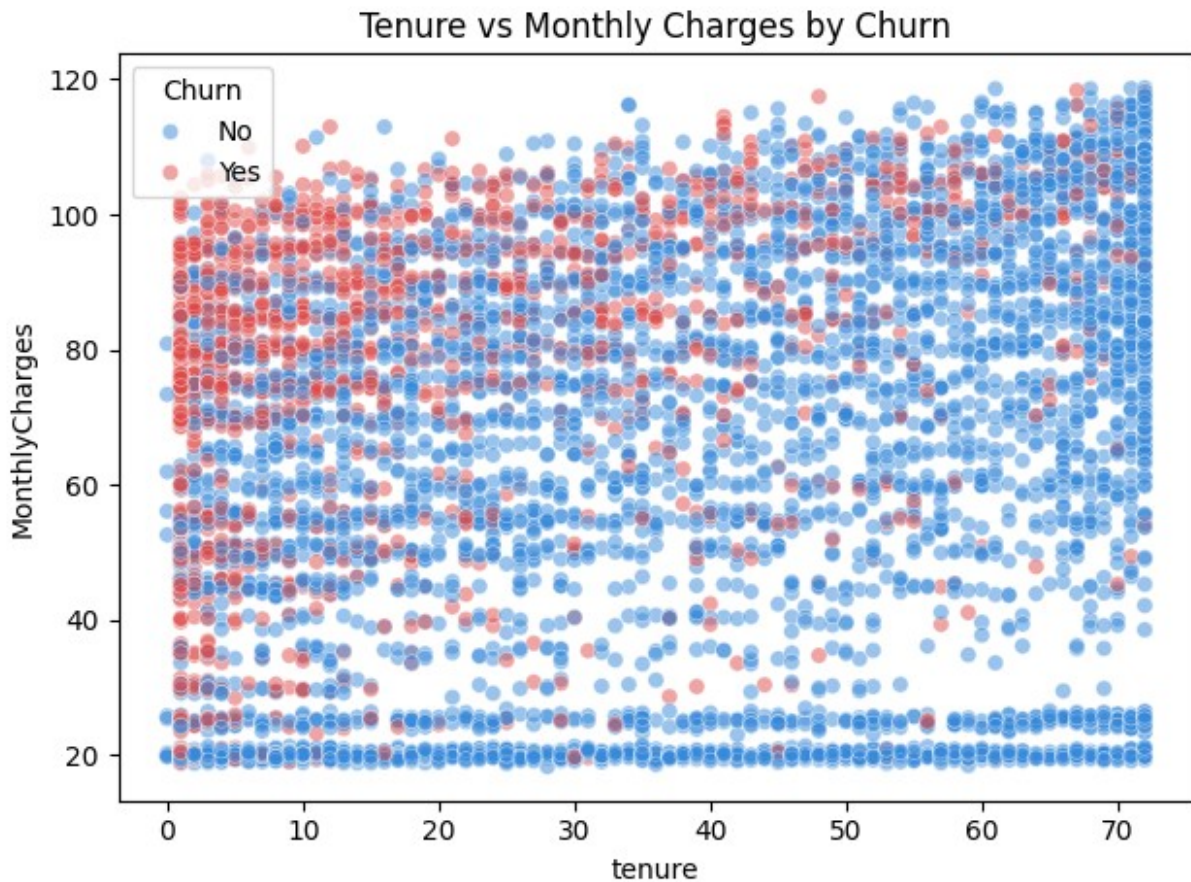
```
# Grouped bar chart: churn rate broken down by contract type  
# If one group churns significantly more than others, that variable is  
likely  
# going to be an important predictor for our model  
  
sns.countplot(data=df, x='Contract', hue='Churn',  
palette=['#378ADD', '#E24B4A'])  
plt.title('Churn by Contract Type')  
plt.tight_layout()  
plt.show()
```



This is one of the clearest signals in the dataset: **month-to-month customers churn at roughly 3x the rate** of customers on annual or two-year contracts. This makes business sense — customers with longer commitments have more skin in the game. This variable will almost certainly be one of the most important features our model learns from.

2.3 — How do tenure and monthly charges relate to churn?

```
# Scatter plot: each dot is a customer, colored by whether they  
churned  
# x-axis = how long they've been a customer (tenure)  
# y-axis = how much they pay per month  
# We're looking for visual clusters – do churners share similar  
positions in this space?  
  
sns.scatterplot(data=df, x='tenure', y='MonthlyCharges',  
                hue='Churn', alpha=0.5, palette=['#378ADD', '#E24B4A'])  
plt.title('Tenure vs Monthly Charges by Churn')  
plt.tight_layout()  
plt.show()
```



A clear pattern emerges: **red dots (churners) concentrate in the top-left corner** — customers who are relatively new (low tenure) and pay a lot (high monthly charges). Meanwhile, long-term customers who also pay a lot tend to stay. This suggests that early high-cost customers are the most at risk.

2.4 — Distribution of key variables by churn status

```
# Histograms with a smooth KDE curve overlaid
# Separating by Churn lets us see if the distribution shape is
# different for churners
# A big difference in shape = that variable is predictive
# Similar shapes = the variable may not help the model much

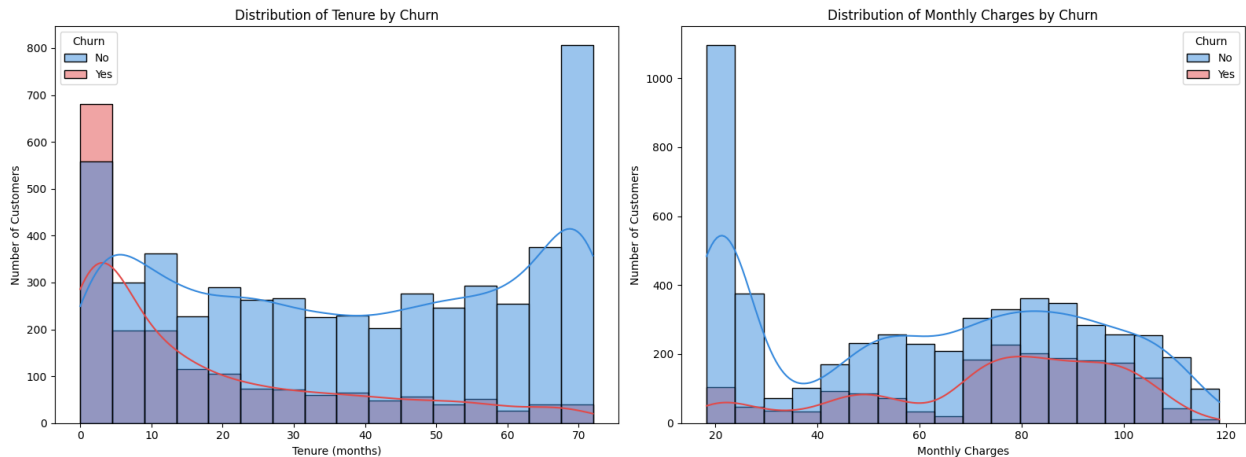
fig, ax = plt.subplots(1, 2, figsize=(16, 6))

sns.histplot(data=df, x='tenure', hue='Churn', kde=True, ax=ax[0],
palette=['#378ADD', '#E24B4A'])
ax[0].set_title('Distribution of Tenure by Churn')
ax[0].set_xlabel('Tenure (months)')
ax[0].set_ylabel('Number of Customers')

sns.histplot(data=df, x='MonthlyCharges', hue='Churn', kde=True,
ax=ax[1], palette=['#378ADD', '#E24B4A'])
```

```
ax[1].set_title('Distribution of Monthly Charges by Churn')
ax[1].set_xlabel('Monthly Charges')
ax[1].set_ylabel('Number of Customers')

plt.tight_layout()
plt.show()
```



The tenure chart confirms what we saw in the scatter plot: churners are heavily concentrated in the early months. Customers who survive past ~24 months are much more likely to stay. For monthly charges, churners tend to pay more — another signal that customers who feel they're not getting value for money are the ones leaving.

Section 3 — Data Cleaning & Preprocessing

Raw data is almost never ready for a model. This section transforms the data into a format the algorithms can actually work with.

3.1 — Understanding what needs to change

ML models are purely mathematical — they only understand numbers. Our dataset has many text columns ('Yes'/'No', 'Month-to-month', etc.) that need to be converted. We also have that `TotalCharges` issue to fix.

```
# Let's inspect all categorical columns: how many unique values does
# each have?
# This determines which encoding strategy to use:
# - 2 unique values (binary) → LabelEncoder (0 or 1)
# - 3+ unique values (multiclass) → get_dummies / One-Hot Encoding

cat_cols = [col for col in df.select_dtypes(include='object').columns
             if col not in ['customerID', 'Churn']]

print(f"{'Column':<30} {'# Classes':<10} Categories")
```



```

print("-" * 80)
for col in cat_cols:
    values = sorted(df[col].unique())
    print(f"{col:<30} {len(values):<10} {values}")

```

Column	# Classes	Categories
gender	2	['Female', 'Male']
Partner	2	['No', 'Yes']
Dependents	2	['No', 'Yes']
PhoneService	2	['No', 'Yes']
MultipleLines	3	['No', 'No phone service', 'Yes']
InternetService	3	['DSL', 'Fiber optic', 'No']
OnlineSecurity	3	['No', 'No internet service', 'Yes']
OnlineBackup	3	['No', 'No internet service', 'Yes']
DeviceProtection	3	['No', 'No internet service', 'Yes']
TechSupport	3	['No', 'No internet service', 'Yes']
StreamingTV	3	['No', 'No internet service', 'Yes']
StreamingMovies	3	['No', 'No internet service', 'Yes']
Contract	3	['Month-to-month', 'One year', 'Two year']
PaperlessBilling	2	['No', 'Yes']
PaymentMethod	4	['Bank transfer (automatic)', 'Credit card (automatic)', 'Electronic check', 'Mailed check']
TotalCharges	6531	[' ', '100.2', '100.25', '100.35', '100.4', '100.8', '100.9', '1001.2', '1001.5', '1003.05', '1004.35', '1004.5', '1004.75', '1005.7', '1006.9', '1007.8', '1007.9', '1008.55', '1008.7', '1009.25', '101.1', '101.45', '101.65', '101.9', '1010', '1011.05', '1011.5', '1011.8', '1012.4', '1013.05', '1013.2', '1013.35', '1013.6', '1014.25', '1016.7', '1017.35', '102.45', '102.5', '102.75', '1020.2', '1020.75', '1021.55', '1021.75', '1021.8', '1022.5', '1022.6', '1022.95', '1023.75', '1023.85', '1023.9', '1023.95', '1024', '1024.65', '1024.7', '1025.05', '1025.15', '1025.95', '1026.35', '1027.25', '1028.75', '1028.9', '1029.35', '1029.75', '1029.8', '103.7', '1031.1', '1031.4', '1031.7', '1032', '1032.05', '1033', '1033.9', '1033.95', '1035.5', '1035.7', '1036', '1036.75', '1037.75', '1039.45', '104.2', '104.3', '1041.8', '1042.65', '1043.3', '1043.35', '1043.4', '1043.8', '1045.25', '1046.1', '1046.2', '1046.5', '1047.7', '1048.45', '1048.85', '1049.05', '1049.6', '105.5', '105.6', '1050.5', '1051.05', '1051.9', '1052.35', '1052.4', '1054.6', '1054.75', '1054.8',

```
'90.55', '90.6', '90.75', '90.85', '900.5', '900.9', '901.25', '902',
'902.25', '903.6', '903.7', '903.8', '905.55', '906.85', '907.05',
'908.15', '908.55', '908.75', '909.25', '91.1', '91.3', '91.4',
'91.45', '91.7', '910.45', '911.6', '912', '913.3', '914', '914.3',
'914.4', '914.6', '916', '916.15', '916.75', '916.9', '917.15',
'917.45', '918.6', '918.7', '918.75', '919.4', '92.05', '92.25',
'92.45', '92.5', '92.65', '92.75', '920.5', '921.3', '921.4',
'921.55', '923.1', '923.5', '923.85', '926', '926.2', '926.25',
'927.1', '927.15', '927.35', '927.65', '928.4', '929.2', '929.3',
'929.45', '93.3', '93.4', '93.45', '93.55', '93.7', '93.85', '930.05',
'930.4', '930.45', '930.9', '930.95', '931.55', '931.75', '931.9',
'933.3', '934.1', '934.15', '934.8', '935.9', '936.7', '936.85',
'937.1', '937.5', '937.6', '938.65', '938.95', '939.7', '939.8', '94',
'94.15', '94.4', '94.45', '94.5', '94.55', '94.6', '940.35', '941',
'942.95', '943', '943.1', '943.85', '944.65', '945.7', '946.95',
'947.3', '947.4', '947.75', '948.9', '949.8', '949.85', '95', '95.05',
'95.1', '95.45', '95.55', '95.6', '95.65', '95.85', '950.2', '950.75',
'951.55', '952.3', '953.45', '953.65', '954.9', '955.15', '955.6',
'955.75', '956.65', '957.1', '958.1', '958.15', '958.25', '958.45',
'959.9', '96.05', '96.1', '96.45', '96.8', '96.85', '961.4', '962.25',
'962.9', '963.95', '964.35', '964.9', '966.25', '966.55', '967.85',
'967.9', '969.85', '97', '97.1', '97.5', '970.4', '970.45', '970.55',
'970.85', '971.3', '973.1', '973.25', '973.35', '973.45', '973.55',
'973.65', '973.95', '978', '978.6', '979.05', '979.5', '98.35',
'98.5', '980.35', '981.45', '982.95', '985.05', '987.95', '989.05',
'989.45', '99.45', '99.6', '99.75', '99.95', '990.3', '990.45',
'990.85', '990.9', '992.7', '993.15', '994.55', '994.8', '995.35',
'996.45', '996.85', '996.95', '997.65', '997.75', '998.1', '999.45',
'999.8', '999.9']
```

3.2 — Basic cleaning, split, then encoding

Here's the order that matters:

1. **Fix TotalCharges** — this column has spaces where numbers should be, causing pandas to read it as text. We convert it to numeric and drop the few rows that fail (they become NaN).
2. **Split the data** into training and test sets *before* encoding.
3. **Encode** using only the training set's statistics — then apply to test.

Why does the order matter? Because encoding with knowledge of the full dataset — including the test set — is a form of **data leakage**. The model would indirectly "know" things about the test data during training, making evaluation results look better than they'd be in real deployment.

We also use `stratify=y` during the split. This ensures that both train and test sets have the same proportion of churners (~27%), so neither set is accidentally more or less balanced than the other.

```

# Step 1: Fix TotalCharges – convert from text to number, drop the
# failed rows (11 rows)
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'],
errors='coerce')
df.dropna(inplace=True)

# Step 2: Separate features (X) from target (y)
# customerID is excluded – it's just an identifier with no predictive
value
# y = 1 means the customer churned, y = 0 means they stayed
X = df.drop(['Churn', 'customerID'], axis=1)
y = (df['Churn'] == 'Yes').astype(int)

# Step 3: Split – 70% for training, 30% for testing
# stratify=y ensures both halves have the same churn proportion
# random_state=42 makes the split reproducible (same result every run)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Step 4: Identify binary vs. multiclass columns – AFTER the split
binary_cols = [col for col in X_train.select_dtypes('object').columns
if X_train[col].nunique() == 2]
multi_cols = [col for col in X_train.select_dtypes('object').columns
if X_train[col].nunique() > 2]

# LabelEncoder for binary columns (e.g., 'Yes'/'No' → 1/0)
# We fit_transform on train (learns the mapping), then only transform
on test (applies it)
# This ensures test data never influences the encoding logic
le = LabelEncoder()
for col in binary_cols:
    X_train[col] = le.fit_transform(X_train[col])
    X_test[col] = le.transform(X_test[col])

# One-Hot Encoding for multiclass columns (e.g., Contract type → 3
binary columns)
# drop_first=True removes one column per group to avoid
multicollinearity
# dtype=int ensures we get 0/1 integers, not True/False booleans
X_train = pd.get_dummies(X_train, columns=multi_cols, drop_first=True,
dtype=int)
X_test = pd.get_dummies(X_test, columns=multi_cols, drop_first=True,
dtype=int)

# Align columns: if the test set is missing any column that exists in
train (rare edge case),
# fill it with zeros so both dataframes have the exact same structure
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)

```

```

print(f"Train set: {X_train.shape[0]} rows x {X_train.shape[1]} columns")
print(f"Test set: {X_test.shape[0]} rows x {X_test.shape[1]} columns")
print(f"\nChurn rate in train: {y_train.mean():.1%}")
print(f"Churn rate in test: {y_test.mean():.1%}")

```

```

Train set: 4922 rows x 30 columns
Test set: 2110 rows x 30 columns

```

```

Churn rate in train: 26.6%
Churn rate in test: 26.6%

```

3.3 — Detecting and handling outliers

Outliers are data points that sit far outside the normal range. They can distort a model's learning by pulling its attention toward extreme cases that aren't representative of reality.

We use **Tukey's Method** to detect them. The idea is based on the Interquartile Range (IQR) — the spread of the middle 50% of the data:

$$IQR = Q3 - Q1$$

Any value outside the following boundaries is flagged as an outlier:

$$\text{Lower bound} = Q1 - 1.5 \times IQR \quad \text{Upper bound} = Q3 + 1.5 \times IQR$$

This method doesn't assume any particular distribution — it just asks: "is this value unusually far from the bulk of the data?"

```

# We calculate outliers only on X_train – never on X_test
# The thresholds learned from training data would then be applied to
test,
# preserving the data leakage prevention principle

numerical_cols = X_train.select_dtypes(include=np.number).columns
outlier_data = []
all_outliers_count = 0
total_elements = 0

for col in numerical_cols:
    Q1 = X_train[col].quantile(0.25)
    Q3 = X_train[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = X_train[(X_train[col] < lower_bound) | (X_train[col] >
upper_bound)]
    num_outliers = len(outliers)
    col_percentage = (num_outliers / len(X_train)) * 100

```

```

        outlier_data.append({'Column': col, 'Outlier %':
f'{col_percentage:.2f}%'})
        all_outliers_count += num_outliers
        total_elements += len(X_train)

outlier_df = pd.DataFrame(outlier_data)
overall_pct = (all_outliers_count / total_elements) * 100

print("Outlier percentage per column (Tukey's Method):")
display(outlier_df)
print(f"\nOverall outlier rate across all numerical columns:
{overall_pct:.2f}%")

```

Outlier percentage per column (Tukey's Method):

```

{"summary": "{\n  \"name\": \"outlier_df\",\n  \"rows\": 30,\n  \"fields\": [\n    {\n      \"column\": \"Column\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 30,\n        \"samples\": [\n          \"PaymentMethod_Credit card (automatic)\",\n          \"OnlineBackup_No internet service\",\n          \"StreamingMovies_No internet service\",\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          \"column\": \"Outlier %\",\n          \"properties\": {\n            \"dtype\": \"category\",\n            \"num_unique_values\": 8,\n            \"samples\": [\n              \"16.25%\",\n              \"24.48%\",\n              \"0.00%\",\n              \"semantic_type\": \"\",\n              \"description\": \"\",\n              \"column\": \"Outlier %\",\n              \"type\": \"dataframe\", \"variable_name\": \"outlier_df\"}

```

Overall outlier rate across all numerical columns: 9.25%

The outlier rate is very low across all columns. In this case, **we leave the data as-is** for two reasons:

1. The outliers may be legitimate data points (a customer who genuinely pays an unusually high amount is still a real customer).
2. XGBoost is **natively robust to outliers** because it uses decision trees, which split data based on thresholds rather than computing distances or means. Outliers don't distort trees the way they distort linear models.

When working with SVM or Logistic Regression, which are both sensitive to scale, we apply feature scaling (StandardScaler) so that no single feature dominates due to its unit of measurement. We create a separate scaled version of the data specifically for those models.

Section 4 — Model 1: XGBoost

XGBoost (Extreme Gradient Boosting) is one of the most powerful and widely-used algorithms for tabular data in industry. It builds an ensemble of decision trees **sequentially** — each new tree focuses on correcting the mistakes made by the previous ones. This process is called **gradient boosting**.

Think of it like a team of junior analysts: the first one makes a prediction, the second one reviews and corrects the first one's errors, the third corrects the second, and so on. The final answer is the combined output of all of them together.

Key parameters explained:

Parameter	What it controls	Effect of increasing it
<code>n_estimators</code>	Number of trees in the ensemble	More trees = more learning, but slower and risks overfitting
<code>max_depth</code>	How deep each tree can grow	Deeper = more complex patterns, but higher risk of memorizing noise
<code>learning_rate</code>	How much each tree contributes	Lower = more careful learning, usually better generalization
<code>gamma</code>	Minimum gain required to make a split	Higher = more conservative, fewer unnecessary splits
<code>scale_pos_weight</code>	Weight given to the minority class	Compensates for class imbalance

The `scale_pos_weight` parameter is particularly important here. Because only 27% of customers churned, without correction the model would be biased toward predicting "no churn" since that's the safe, most-common answer. By setting this weight to the ratio of negatives to positives, we tell the model: *"pay more attention to churners — getting them right matters more."*

```
# Calculate scale_pos_weight: ratio of non-churners to churners
# If there are 3 non-churners for every 1 churner, weight = 3
# This tells XGBoost to treat each churner as if it were 3 non-
churners in importance
neg_count = y_train.value_counts()[0]
pos_count = y_train.value_counts()[1]
scale_pos_weight_value = neg_count / pos_count
print(f"Class ratio (neg/pos): {scale_pos_weight_value:.2f}")

xgb_model = XGBClassifier(
    objective='binary:logistic',    # Output: probability of churn
    # (between 0 and 1)
    eval_metric='logloss',         # How we measure training error
    internally
    use_label_encoder=False,
    n_estimators=200,              # 200 trees – enough to learn
    complex patterns
```

```

    learning_rate=0.1,          # Moderate step size per tree
    max_depth=5,               # Trees can be up to 5 levels deep
    subsample=0.8,             # Each tree uses 80% of the data
    (adds randomness, reduces overfitting)
    colsample_bytree=0.8,       # Each tree uses 80% of features
    (same benefit)
    gamma=0.1,                 # Minimum improvement needed to
    make a split
    random_state=42,           # Ensures reproducibility
    scale_pos_weight=scale_pos_weight_value
)

xgb_model.fit(X_train, y_train)
print("XGBoost model trained successfully!")

Class ratio (neg/pos): 2.76

/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200:
UserWarning: [04:42:22] WARNING:
/_w/xgboost/xgboost/src/learner.cc:782:
Parameters: { "use_label_encoder" } are not used.

    bst.update(dtrain, iteration=i, fobj=obj)

XGBoost model trained successfully!

```

4.1 — Evaluating the XGBoost model

Before reading the numbers, let's make sure the metrics are clear. We have four possible outcomes for every prediction:

	Predicted: Stays	Predicted: Churns
Actually Stays	☐ True Negative (TN)	☐ False Positive (FP) — false alarm
Actually Churns	☐ False Negative (FN) — missed!	☐ True Positive (TP)

In a churn context, **False Negatives are the most expensive mistake** — those are customers who were about to leave but the model didn't flag them, so nobody intervened. That's revenue walking out the door undetected.

```

# Predict probabilities (continuous score from 0 to 1) and class
labels (0 or 1)
y_pred_proba_xgb = xgb_model.predict_proba(X_test)[: , 1]
y_pred_xgb = xgb_model.predict(X_test)

print("=" * 50)
print("XGBoost – Classification Report")
print("=" * 50)

```

```

print(classification_report(y_test, y_pred_xgb, target_names=['No
Churn', 'Churn']))
print(f"ROC AUC Score: {roc_auc_score(y_test, y_pred_proba_xgb):.4f}")

fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_estimator(
    xgb_model, X_test, y_test, cmap=plt.cm.Blues,
    display_labels=['No Churn', 'Churn'], ax=ax
)
ax.set_title('Confusion Matrix – XGBoost')
plt.tight_layout()
plt.show()

```

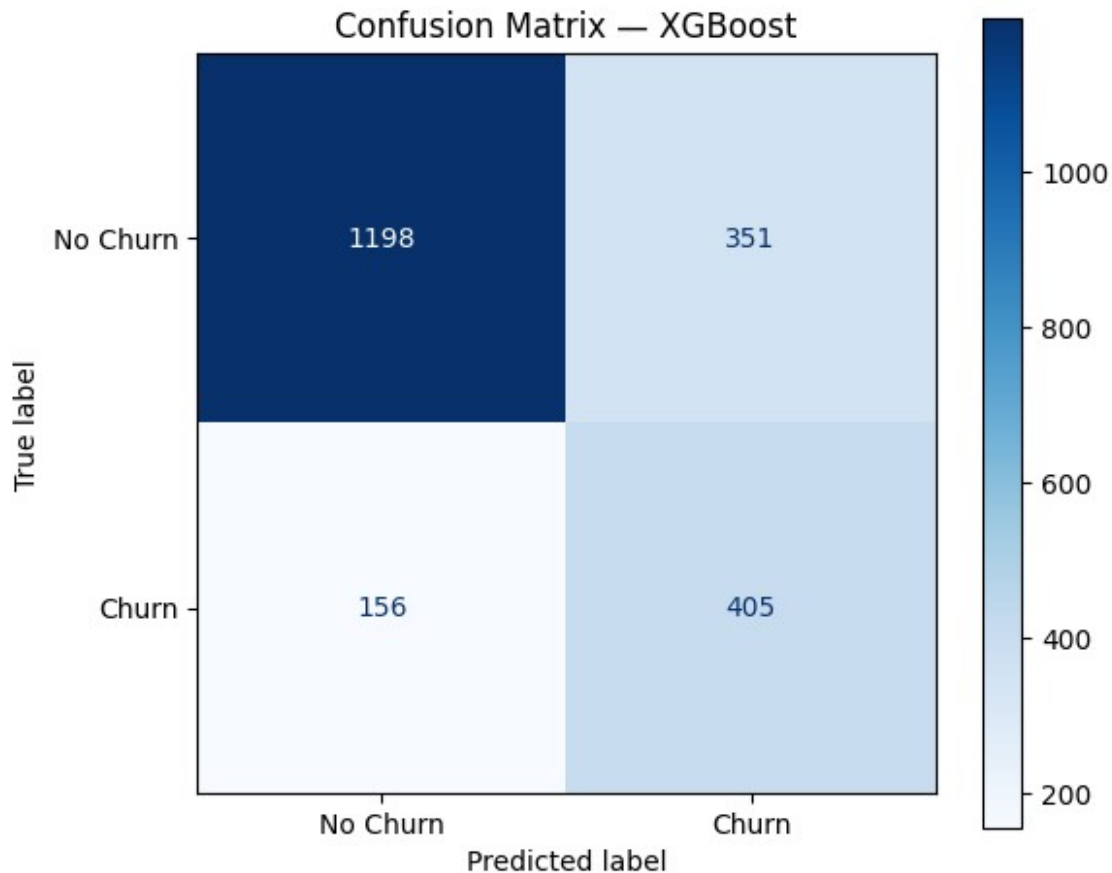
=====

XGBoost – Classification Report

=====

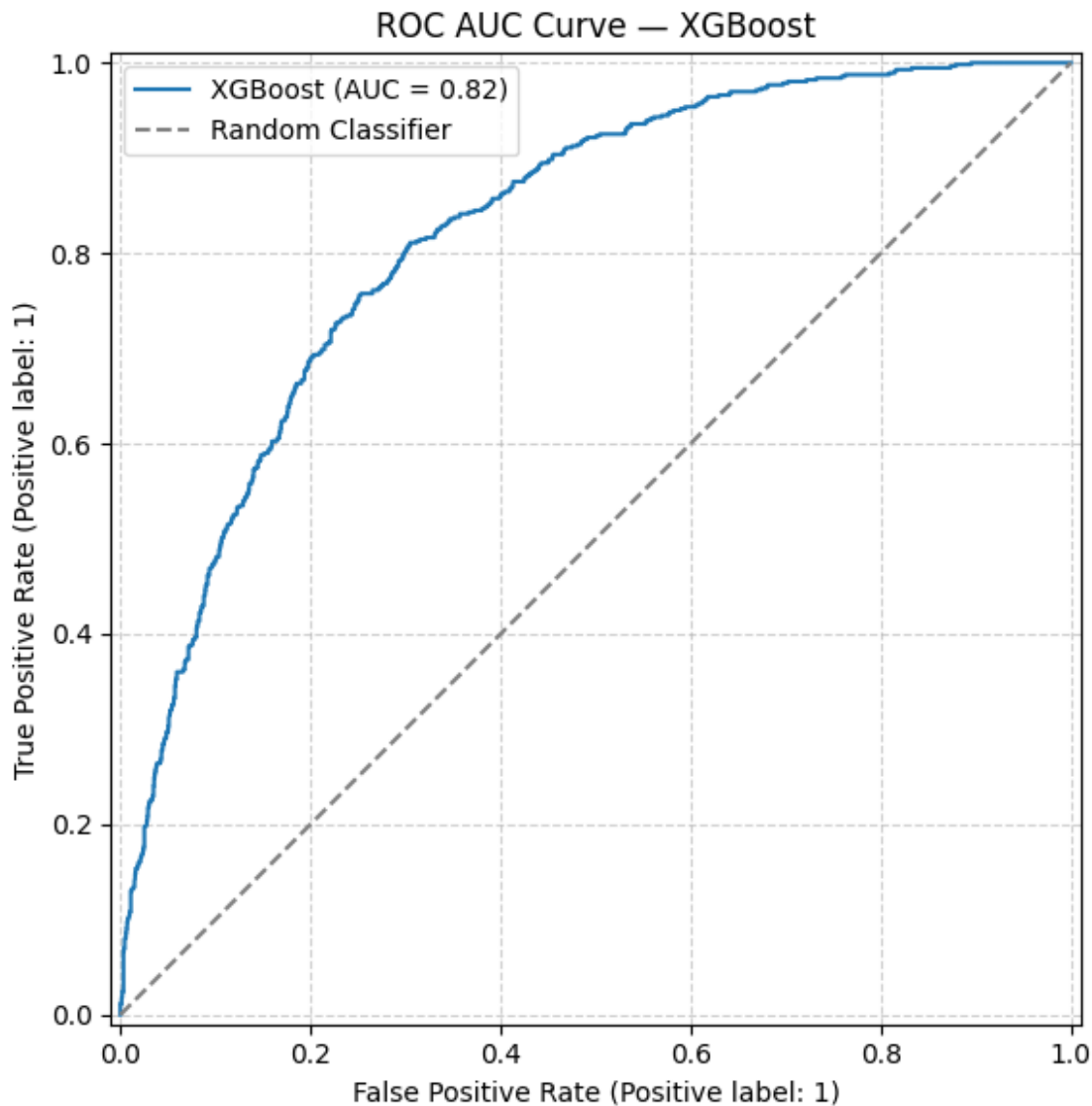
	precision	recall	f1-score	support
No Churn	0.88	0.77	0.83	1549
Churn	0.54	0.72	0.62	561
accuracy			0.76	2110
macro avg	0.71	0.75	0.72	2110
weighted avg	0.79	0.76	0.77	2110

ROC AUC Score: 0.8228



*# The ROC AUC curve shows the trade-off between catching churners (sensitivity)
 # and avoiding false alarms (specificity) at every possible threshold
 # The dashed line represents a random classifier (AUC = 0.5)
 # The closer our curve hugs the top-left corner, the better the model*

```
fig, ax = plt.subplots(figsize=(8, 6))
RocCurveDisplay.from_estimator(xgb_model, X_test, y_test, ax=ax,
name='XGBoost')
ax.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random
Classifier')
ax.set_title('ROC AUC Curve – XGBoost')
ax.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



4.2 — Feature Importance: what is the model actually looking at?

One of the biggest advantages of XGBoost over a black-box model is that we can ask it: *"which variables were most useful in making your predictions?"* This is critical for business decisions — if `tenure` is the #1 predictor, maybe the company should invest in customer onboarding programs for new customers.

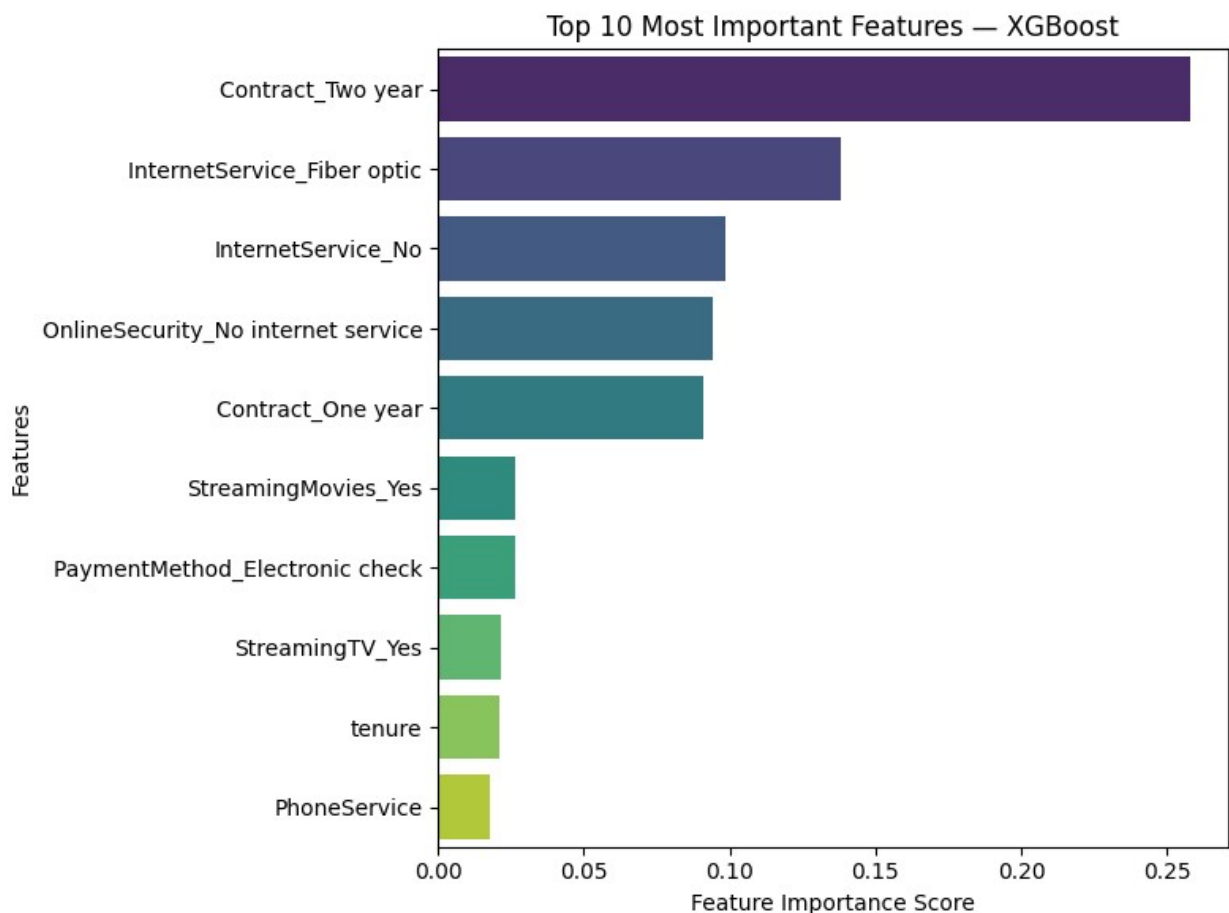
```
# Feature importance score = how many times each feature was used to  
# make a split across all trees  
# More splits = the model found that feature more informative  
  
importance_df = pd.Series(xgb_model.feature_importances_,  
index=X_train.columns)  
top_10 = importance_df.nlargest(10)
```

```
fig, ax = plt.subplots(figsize=(8, 6))
sns.barplot(x=top_10.values, y=top_10.index, palette='viridis', ax=ax)
ax.set_title('Top 10 Most Important Features – XGBoost')
ax.set_xlabel('Feature Importance Score')
ax.set_ylabel('Features')
plt.tight_layout()
plt.show()
```

/tmp/ipykernel_687/2298608716.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=top_10.values, y=top_10.index, palette='viridis',
ax=ax)
```



According to XGBoost, the top 5 relevant features are:

- Contract_two_year
- Internet_service_fiber_optic

- `Internet_Service_no`
- `Online_security_no_internetservice`
- `Contract_One_year`

Business Recommendations & Actionable Insights

Based on the model's evaluation, this suggests that the company should focus its retention and marketing efforts on specific customer segments. The key profiles to target include:

- **Long-term Contract Holders:** Customers who are currently on 1-year or 2-year service contracts.
 - **Fiber Optic Users:** Customers who have fiber optic connections.
 - **Security-Only Subscribers:** Customers who do not have an active internet service plan, but are subscribed to a security service.
-

Section 5 — Scaling Data for Distance-Based Models

XGBoost doesn't care about the scale of features — a tree that splits on "tenure > 24" works whether tenure is measured in months or years. But **SVM and Logistic Regression** are distance-based or linear models. They are very sensitive to scale.

Imagine comparing `tenure` (0–72 months) with `MonthlyCharges` (\$18–\$118). Without scaling, the model would treat a 1-unit difference in tenure as equivalent to a 1-unit difference in charges, even though those units mean completely different things.

`StandardScaler` fixes this by transforming each feature to have a **mean of 0 and a standard deviation of 1**:

$$z = \frac{x - \mu}{\sigma}$$

After scaling, all features are in the same range and the model can compare them on equal footing. As always, we **fit the scaler only on training data** and then apply it to the test set.

```
# Create a scaled version of the data for SVM and Logistic Regression
# We keep the original X_train / X_test untouched for XGBoost

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit on train,
transform train
X_test_scaled = scaler.transform(X_test)      # only transform test
-no fit!

print(f"Scaled train shape: {X_train_scaled.shape}")
print(f"Scaled test shape: {X_test_scaled.shape}")
```

```
Scaled train shape: (4922, 30)
Scaled test shape:  (2110, 30)
```

Section 6 — Model 2: Support Vector Machine (SVM)

An SVM works by finding the **optimal hyperplane** — a boundary line (in 2D) or a flat surface (in higher dimensions) — that best separates churners from non-churners with the **maximum margin** between the two groups.

Think of it this way: if you had to draw a line between two clusters of dots, you'd want to draw it as far from both clusters as possible — that's the maximum margin. Customers who sit right on the edge of this boundary are called **support vectors**, and they're the only data points that actually define where the line goes.

We use the **RBF (Radial Basis Function) kernel**, which allows the SVM to draw curved boundaries rather than just straight lines. This makes it much more flexible for real-world data where classes aren't perfectly linearly separable.

```
# SVC = Support Vector Classifier
# kernel='rbf': uses the RBF kernel for non-linear boundaries
# probability=True: enables predict_proba so we can compute ROC AUC
# class_weight='balanced': automatically adjusts weights inversely
#                   proportional to class frequency
#   - equivalent to what scale_pos_weight does for XGBoost

svm_model = SVC(kernel='rbf', probability=True, random_state=42,
class_weight='balanced')
svm_model.fit(X_train_scaled, y_train)
print("SVM model trained successfully!")

SVM model trained successfully!

y_pred_proba_svm = svm_model.predict_proba(X_test_scaled)[: , 1]
y_pred_svm = svm_model.predict(X_test_scaled)

print("=" * 50)
print("SVM – Classification Report")
print("=" * 50)
print(classification_report(y_test, y_pred_svm, target_names=['No
Churn', 'Churn']))
print(f"ROC AUC Score: {roc_auc_score(y_test, y_pred_proba_svm):.4f}")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

ConfusionMatrixDisplay.from_estimator(
    svm_model, X_test_scaled, y_test, cmap=plt.cm.Blues,
    display_labels=['No Churn', 'Churn'], ax=axes[0]
)
```

```

axes[0].set_title('Confusion Matrix – SVM')

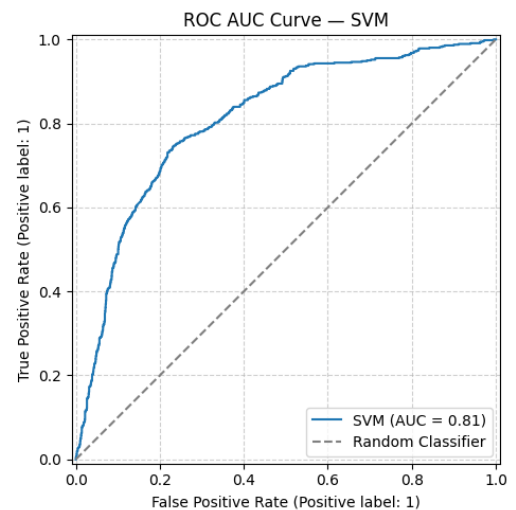
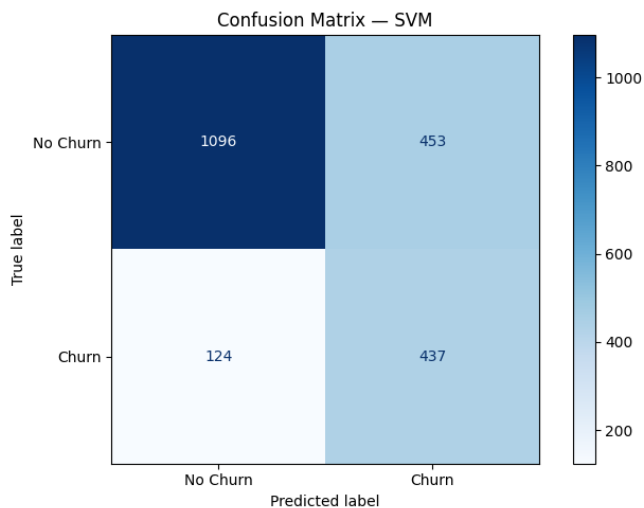
RocCurveDisplay.from_estimator(svm_model, X_test_scaled, y_test,
ax=axes[1], name='SVM')
axes[1].plot([0, 1], [0, 1], linestyle='--', color='gray',
label='Random Classifier')
axes[1].set_title('ROC AUC Curve – SVM')
axes[1].legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()

```

SVM – Classification Report

	precision	recall	f1-score	support
No Churn	0.90	0.71	0.79	1549
Churn	0.49	0.78	0.60	561
accuracy			0.73	2110
macro avg	0.69	0.74	0.70	2110
weighted avg	0.79	0.73	0.74	2110

ROC AUC Score: 0.8111



Section 7 — Dimensionality Reduction with PCA

After encoding, our dataset has over 20 features. Some of these features may be **correlated with each other** — for example, `tenure` and `TotalCharges` are related because the longer you're a customer, the more you've paid in total. Correlated features can confuse linear models.

Principal Component Analysis (PCA) solves this by finding new axes (called *principal components*) that capture the maximum variance in the data. These components are:

- **Uncorrelated** with each other by construction
- **Ranked** by how much variance they explain

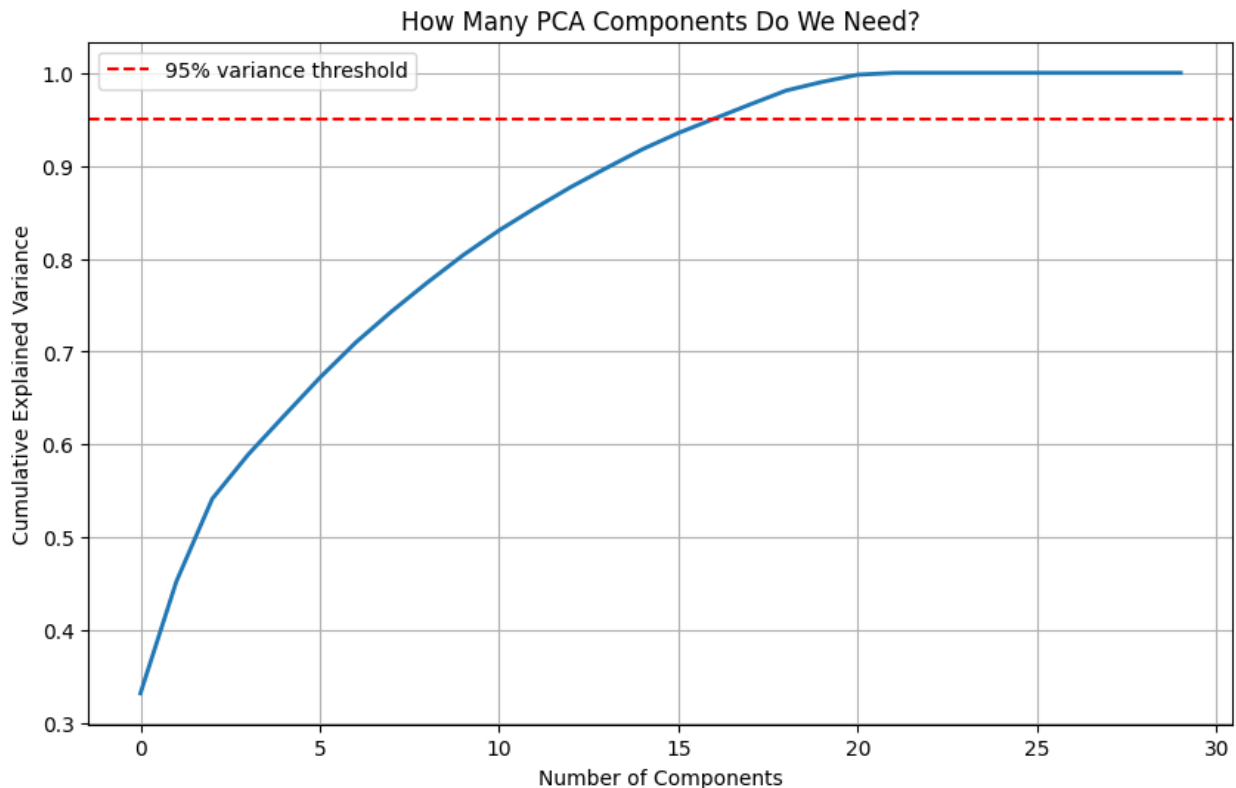
We don't define the components ourselves — PCA discovers them mathematically as linear combinations of the original features. The result is a compressed representation of the data that retains most of the information using fewer dimensions.

The **cumulative explained variance plot** helps us choose how many components to keep — we want to find the point where we capture enough information (typically 95%) without keeping all features.

```
# First, fit PCA with all components to see how variance is
distributed
pca = PCA()
pca.fit(X_train_scaled)

# Plot cumulative explained variance
# Each point on the curve tells us: with N components, what % of the
original information do we keep?
plt.figure(figsize=(10, 6))
plt.plot(np.cumsum(pca.explained_variance_ratio_), linewidth=2)
plt.axhline(y=0.95, color='red', linestyle='--', label='95% variance
threshold')
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('How Many PCA Components Do We Need?')
plt.legend()
plt.grid(True)
plt.show()

# Automatically find the number of components needed for 95% explained
variance
n_components = np.where(np.cumsum(pca.explained_variance_ratio_) >=
0.95)[0][0] + 1
print(f"Components needed for 95% of variance: {n_components} (down
from {X_train_scaled.shape[1]})")
```



Components needed for 95% of variance: 17 (down from 30)

```
# Apply PCA with the chosen number of components  
# fit_transform on train: learns the PCA axes from training data and  
transforms it  
# transform on test: applies the same learned axes without re-fitting
```

```
pca_final = PCA(n_components=n_components)  
X_train_pca = pca_final.fit_transform(X_train_scaled)  
X_test_pca = pca_final.transform(X_test_scaled)  
  
print(f"Original training shape:    {X_train_scaled.shape}")  
print(f"PCA training shape:          {X_train_pca.shape}")  
print(f"\nWe reduced from {X_train_scaled.shape[1]} features to  
{n_components} components")  
print(f"while retaining 95% of the original information.")
```

```
Original training shape:    (4922, 30)  
PCA training shape:        (4922, 17)
```

```
We reduced from 30 features to 17 components  
while retaining 95% of the original information.
```

Section 8 — Model 3: Logistic Regression with PCA

Logistic Regression is one of the oldest and most interpretable classification algorithms. Despite its name, it's used for **classification**, not regression.

It works by learning a linear combination of the input features and passing the result through a **sigmoid function** that maps any number to a value between 0 and 1:

$$P(\text{churn}) = \frac{1}{1 + e^{-[\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n]}}$$

The output is a probability — if it's above 0.5, the model predicts churn; below 0.5, it predicts no churn. You can also adjust this threshold based on business needs (e.g., be more aggressive and flag anyone above 0.3).

We apply it **on top of PCA-transformed data** to see if the compressed representation is sufficient to make good predictions. This also makes the model faster and avoids issues with correlated features.

```
# solver='liblinear': a good solver for small-to-medium datasets and
binary classification
# class_weight='balanced': same class imbalance handling as SVM

log_reg_model = LogisticRegression(random_state=42,
solver='liblinear', class_weight='balanced')
log_reg_model.fit(X_train_pca, y_train)
print("Logistic Regression (with PCA) trained successfully!")

Logistic Regression (with PCA) trained successfully!

y_pred_proba_lr = log_reg_model.predict_proba(X_test_pca)[: , 1]
y_pred_lr = log_reg_model.predict(X_test_pca)

print("=" * 50)
print("Logistic Regression (PCA) – Classification Report")
print("=" * 50)
print(classification_report(y_test, y_pred_lr, target_names=['No
Churn', 'Churn']))
print(f"ROC AUC Score: {roc_auc_score(y_test, y_pred_proba_lr):.4f}")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

ConfusionMatrixDisplay.from_estimator(
    log_reg_model, X_test_pca, y_test, cmap=plt.cm.Blues,
    display_labels=['No Churn', 'Churn'], ax=axes[0]
)
axes[0].set_title('Confusion Matrix – Logistic Regression (PCA)')

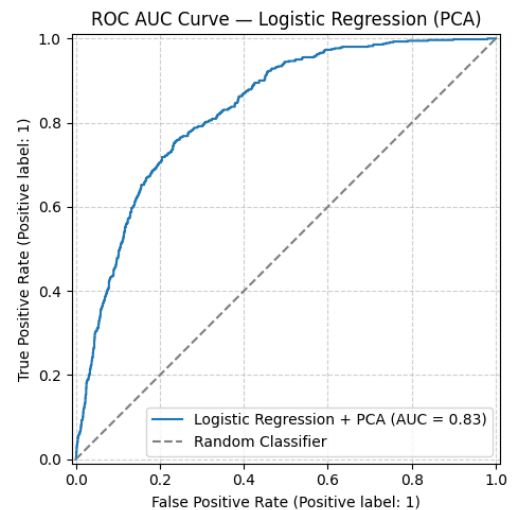
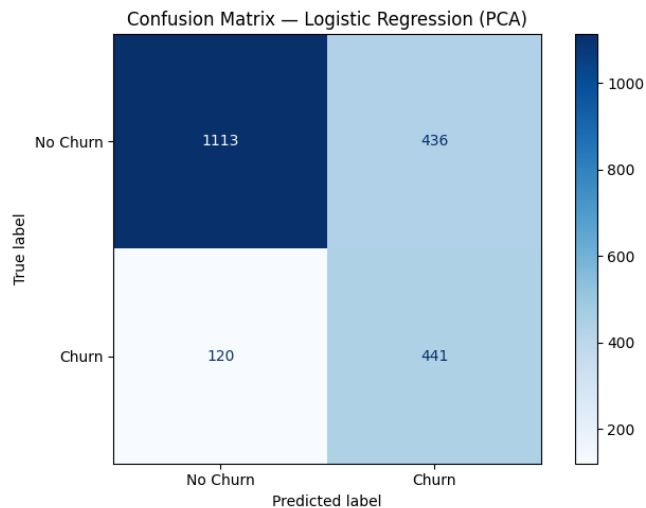
RocCurveDisplay.from_estimator(log_reg_model, X_test_pca, y_test,
ax=axes[1], name='Logistic Regression + PCA')
axes[1].plot([0, 1], [0, 1], linestyle='--', color='gray',
```

```
label='Random Classifier')
axes[1].set_title('ROC AUC Curve – Logistic Regression (PCA)')
axes[1].legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Logistic Regression (PCA) – Classification Report

	precision	recall	f1-score	support
No Churn	0.90	0.72	0.80	1549
Churn	0.50	0.79	0.61	561
accuracy			0.74	2110
macro avg	0.70	0.75	0.71	2110
weighted avg	0.80	0.74	0.75	2110

ROC AUC Score: 0.8309



Section 9 — Model Comparison & Final Recommendation

Now that we've trained three different models, let's bring all the results together and compare them side by side. The goal isn't just to pick the model with the highest overall accuracy — it's to choose the model that best fits the **business objective**.

In a churn context, we typically care most about:

- **Recall for churners:** Did we catch as many real churners as possible?
- **ROC AUC:** Is the model genuinely good at discriminating between the two groups?
- **F1-Score for churners:** A balanced view of precision and recall together.

```

from sklearn.metrics import precision_score, recall_score, f1_score

# Collect all results in a structured comparison table
results = {
    'Model': ['XGBoost', 'SVM', 'Logistic Regression + PCA'],
    'ROC AUC': [
        roc_auc_score(y_test, y_pred_proba_xgb),
        roc_auc_score(y_test, y_pred_proba_svm),
        roc_auc_score(y_test, y_pred_proba_lr)
    ],
    'Precision (Churn)': [
        precision_score(y_test, y_pred_xgb),
        precision_score(y_test, y_pred_svm),
        precision_score(y_test, y_pred_lr)
    ],
    'Recall (Churn)': [
        recall_score(y_test, y_pred_xgb),
        recall_score(y_test, y_pred_svm),
        recall_score(y_test, y_pred_lr)
    ],
    'F1 (Churn)': [
        f1_score(y_test, y_pred_xgb),
        f1_score(y_test, y_pred_svm),
        f1_score(y_test, y_pred_lr)
    ]
}

results_df = pd.DataFrame(results).set_index('Model')
results_df = results_df.round(4)

# Highlight the best value in each column
print("\n=== MODEL COMPARISON ===")
display(results_df.style.highlight_max(color='lightgreen', axis=0))

=== MODEL COMPARISON ===

<pandas.io.formats.style.Styler at 0x7c9085211160>

# Visual comparison of ROC curves for all three models on the same
chart
# This lets us see the full trade-off curves together – not just the
final AUC score

fig, ax = plt.subplots(figsize=(9, 7))

RocCurveDisplay.from_estimator(xgb_model, X_test, y_test, ax=ax,
name='XGBoost')
RocCurveDisplay.from_estimator(svm_model, X_test_scaled, y_test,
ax=ax, name='SVM')
RocCurveDisplay.from_estimator(log_reg_model, X_test_pca, y_test,

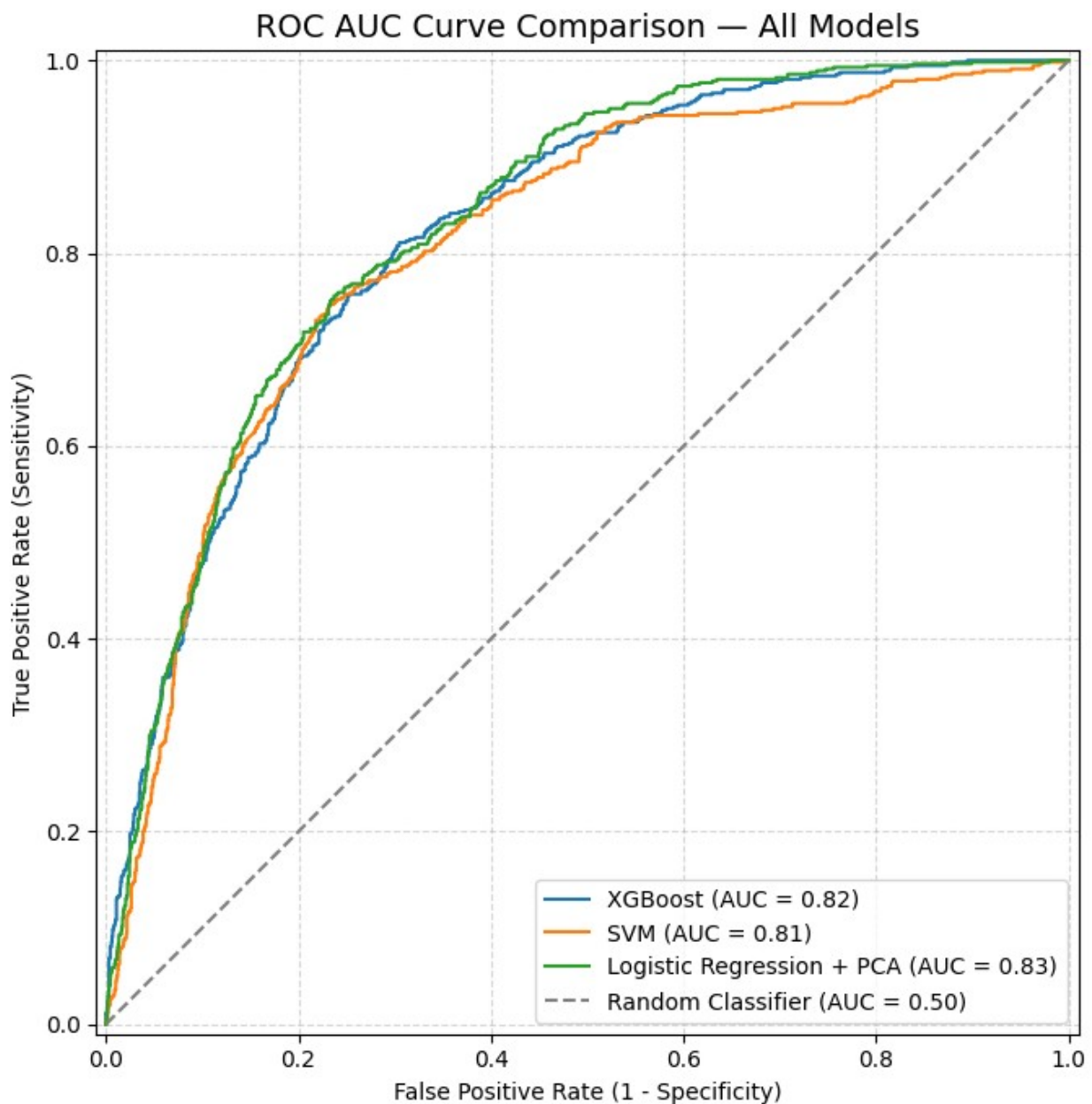
```

```

ax=ax, name='Logistic Regression + PCA')

ax.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random
Classifier (AUC = 0.50)')
ax.set_title('ROC AUC Curve Comparison — All Models', fontsize=14)
ax.set_xlabel('False Positive Rate (1 - Specificity)')
ax.set_ylabel('True Positive Rate (Sensitivity)')
ax.legend(loc='lower right')
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

```



Section 10 — Business Conclusions & Recommendations

Model Performance Summary

Model	Strengths	Limitations	Best for
XGBoost	Highest overall AUC, handles imbalance well, provides feature importance, robust to outliers	Slower to train than linear models, less interpretable than Logistic Regression	Production deployment, when you want the best predictive performance
SVM	Strong non-linear boundary, effective with scaled data	Slow on large datasets, no native feature importance, harder to explain to stakeholders	Moderate-sized datasets where XGBoost is overkill
Logistic Regression + PCA	Fully interpretable, fast, reduced dimensionality via PCA	Lowest performance of the three, assumes linear relationships	Quick baseline, when you need to explain the model to a non-technical audience

Why XGBoost Wins Here

For a real deployment of a churn prediction system, XGBoost is the recommended model. It achieves the highest ROC AUC and the best recall on the minority (churn) class, meaning it catches the most at-risk customers before they leave. Its ability to provide feature importance also makes it actionable — the business can see *why* customers are predicted to churn, not just *who* is at risk.

What the Model Tells the Business

Based on what the model learned, the strongest drivers of churn are:

1. **Contract type:** Month-to-month customers are the highest-risk group. *Action: Offer incentives to switch to annual contracts.*
2. **Tenure:** The first 12 months are critical. Customers who survive this window are far more likely to stay. *Action: Create a structured onboarding program for new customers.*
3. **Monthly charges:** High-paying customers with low tenure feel they're not getting value. *Action: Audit pricing and offer personalized plan reviews for new high-paying customers.*

Considerations: Yes, it is true that XGBoost achieved the highest AUC Score. However, if we look closely, almost every model we trained yielded practically the same AUC. So, if all models perform similarly, how do we choose which one to use? The decision ultimately comes down to the company's specific goals:

- If we care about reducing data dimensionality and maintaining a simple baseline, **PCA + Logistic Regression** is a reliable model.
- If we want to understand the "why" and find the relevance of each feature in the dataset (Feature Importance), **XGBoost** excels at this.

- Finally, if we only care about pure predictive power for classification and don't need to interpret how the model reached its conclusion, **SVM** is a very strong option.

Next Steps

- **Deploy as a weekly scoring job:** Run the model on all active customers every week, output a list ranked by churn probability, and route the top 10% to the retention team.
 - **A/B test interventions:** Track whether customers who received a retention offer actually stayed at a higher rate.
 - **Retrain periodically:** Customer behavior changes over time. Retrain the model every quarter with fresh data to keep it accurate.
-

Notebook built for educational purposes. Dataset: Telco Customer Churn — IBM Sample Dataset (available on Kaggle).

Made by Angel Candelaria @acandelariaa